Project Documentation

File:	execute.project
Date:	04/05/2026
Profile:	CODESYS V3.5 SP21 Patch 5

Table of contents

1 Device: Device 3
1.1 PLC Logic: Plc Logic 3
1.1.1 Application: Application 3
2 : Project Settings 30

1 Device: Device

Users and Groups

Users:

Groups

Access Rights

View
Modify
Execute
Add/remove children

Symbol Rights

Information

Name:	CODESYS Control Win V3 x64
Vendor:	CODESYS
Categories:	PLCs
Type:	4096
ID:	0000 0004
Version:	3.5.21.50
Order number:	???
Description:	CODESYS V3 Soft-PLC for Windows with non realtime capabilities (CODESYS Control Win V3 x64)

1.1 PLC Logic: Plc Logic

1.1.1 Application: Application

1.1.1.1 Global Variable List: GVL_IO

```
1   VAR_GLOBAL
2       Start_PB              :  BOOL ;
3       Stop_PB               :  BOOL ;
4       Reset_PB              :  BOOL ;
5       Abort_PB              :  BOOL ;
6
7       Infeed_Prox           :  BOOL ;
8       Fill_Pos_Sensor       :  BOOL ;
9       Level_Full_Sensor     :  BOOL ;
10      Red_Container_Sensor  :  BOOL ;
11      Blue_Container_Sensor :  BOOL ;
12      Exit_Sensor           :  BOOL ;
13      Robot_Pick_Done       :  BOOL ;
14
15      Conveyor_Fault        :  BOOL ;
16      Low_Fluid_A           :  BOOL ;
17      Low_Fluid_B           :  BOOL ;
18      Infeed_Block_Sensor   :  BOOL ;
19      Outfeed_Block_Sensor  :  BOOL ;
20
21      Conveyor_Motor        :  BOOL ;
22      Valve_A               :  BOOL ;
23      Valve_B               :  BOOL ;
24      Pickup_Ready          :  BOOL ;
25      Alarm_Lamp            :  BOOL ;
26      Stack_Light_Green     :  BOOL ;
27      Stack_Light_Red       :  BOOL ;
28      Stack_Light_Amber     :  BOOL ;
29
30      SC                    :  BOOL ;
31      BatchDone             :  BOOL ;
32      ClearCmd              :  BOOL ;
33      HoldCmd               :  BOOL ;
34      UnholdCmd             :  BOOL ;
35      SuspendCmd            :  BOOL ;
36      UnsuspendCmd          :  BOOL ;
37      AbortCmd              :  BOOL ;
38
39      ExecStep              :  INT ;
40      ContainerType         :  INT ;
41
42      FaultActive           :  BOOL ;
43      FaultLatched          :  BOOL ;
44      FaultCode             :  INT ;
45  END_VAR
46
```

1.1.1.2 Library Manager: Library Manager

```
#IoStandard
    Base Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
    CmpErrors2 Interfaces, * (System)
    #CmpIoMgr Implementation
        CmpErrors2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpIoMgr Interfaces, * (System)
        Base Interfaces, * (System)
    CmpIoMgr Interfaces, * (System)
#3SLicense
    #CmpLog
        SysTypes2 Interfaces, * (System)
    #CAA Types
        #SysCpuHandling
            #SysMem
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
    #Standard
    #Component Manager
        #CmpEventMgr
            CmpEventMgr Interfaces, * (System)
            #CmpEventMgr Implementation
                CmpEventMgr Interfaces, * (System)
                CmpErrors2 Interfaces, * (System)
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
        #CmpApp
            SysTypes2 Interfaces, * (System)
            #CmpEventMgr
                CmpEventMgr Interfaces, * (System)
                #CmpEventMgr Implementation
                    CmpEventMgr Interfaces, * (System)
                    CmpErrors2 Interfaces, * (System)
                    SysTypes2 Interfaces, * (System)
                SysTypes2 Interfaces, * (System)
                CmpErrors2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
    #CmpApp
        SysTypes2 Interfaces, * (System)
        #CmpEventMgr
            CmpEventMgr Interfaces, * (System)
            #CmpEventMgr Implementation
                CmpEventMgr Interfaces, * (System)
                CmpErrors2 Interfaces, * (System)
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
    #CmpCodeMeter
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
```

```
    CmpErrors2 Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
    #SysCpuHandling
        #SysMem
            SysTypes2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
    #CmpEventMgr
        CmpEventMgr Interfaces, * (System)
        #CmpEventMgr Implementation
            CmpEventMgr Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
#CAA Device Diagnosis
    #IoStandard
        Base Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
        #CmpIoMgr Implementation
            CmpErrors2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpIoMgr Interfaces, * (System)
            Base Interfaces, * (System)
        CmpIoMgr Interfaces, * (System)
    #CAA Types
        #SysCpuHandling
            #SysMem
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
    #CmpEventMgr
        CmpEventMgr Interfaces, * (System)
        #CmpEventMgr Implementation
            CmpEventMgr Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
    IoDriver Interfaces, * (System)
    Base Interfaces, * (System)
    CmpErrors2 Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
    #CmpAsyncMgr
        SysTypes2 Interfaces, * (System)
    #CommFB
        #SysMem
            SysTypes2 Interfaces, * (System)
        IoDriver Profibus2 Interfaces, * (System)
#Standard
#BreakpointLogging
    #CmpLog
        SysTypes2 Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
#IecSfc
    #Standard
```

1.1.1.3 POU: Aborted_ST

```
FUNCTION_BLOCK Aborted_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := TRUE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := FALSE ;
Stack_Light_Red := TRUE ;

SC := FALSE ;

```

1.1.1.4 POU: Aborting_ST

```
FUNCTION_BLOCK Aborting_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := TRUE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := FALSE ;
Stack_Light_Red := TRUE ;

SC := TRUE ;

```

1.1.1.5 POU: Clearing_ST

```
FUNCTION_BLOCK Clearing_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := FALSE ;
Stack_Light_Red := FALSE ;

FaultActive := FALSE ;
FaultLatched := FALSE ;
FaultCode := 0 ;
AbortCmd := FALSE ;

ExecStep := 0 ;
ContainerType := 0 ;

SC := TRUE ;

```

1.1.1.6 POU: Complete_ST

```
FUNCTION_BLOCK Complete_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := TRUE ;
Stack_Light_Red := FALSE ;

SC := FALSE ;
```

1.1.1.7 POU: Completing_ST

```
FUNCTION_BLOCK Completing_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

SC := TRUE ;

```

1.1.1.8 POU: Execute_ST

```
FUNCTION_BLOCK Execute_ST
VAR_INPUT
END_VAR

VAR_OUTPUT
END_VAR

VAR
    T_MoveToFill : TON ;
    T_Fill       : TON ;
    T_MoveToExit : TON ;
    T_Pickup     : TON ;
END_VAR

```

```
// ====================================================
// Execute_ST
// PackML Execute state logic
// This code controls the filling process
// ExecStep is used to divide Execute into smaller steps
// Faults are separated into Hold, Suspend, and Abort groups
// ====================================================


// =========================
// Default Execute status
// =========================

SC := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := TRUE ;
```

```
18      Stack_Light_Amber  :=  FALSE ;
19      Stack_Light_Red  :=  FALSE ;
20
21      FaultActive  :=  FALSE ;
22
23
24      // ========================
25      // Timer logic
26      // Timers stop when Hold, Suspend, Abort, or Stop is active
27      // This prevents timeout faults during pause states
28      // ========================
29
30      T_MoveToFill (
31          IN  :=  ( ExecStep  =  20 )
32                AND  NOT  HoldCmd
33                AND  NOT  SuspendCmd
34                AND  NOT  AbortCmd
35                AND  NOT  Stop_PB ,
36          PT  :=  T#50S
37      ) ;
38
39      T_Fill (
40          IN  :=  ( ( ExecStep  =  40 )  OR  ( ExecStep  =  50 ) )
41                AND  NOT  HoldCmd
42                AND  NOT  SuspendCmd
43                AND  NOT  AbortCmd
44                AND  NOT  Stop_PB ,
45          PT  :=  T#50S
46      ) ;
47
48      T_MoveToExit (
49          IN  :=  ( ExecStep  =  70 )
50                AND  NOT  HoldCmd
51                AND  NOT  SuspendCmd
52                AND  NOT  AbortCmd
53                AND  NOT  Stop_PB ,
54          PT  :=  T#50S
55      ) ;
56
57      T_Pickup (
58          IN  :=  ( ExecStep  =  90 )
59                AND  NOT  HoldCmd
60                AND  NOT  SuspendCmd
61                AND  NOT  AbortCmd
62                AND  NOT  Stop_PB ,
63          PT  :=  T#50S
64      ) ;
65
66
67      // ====================================================
68      // Fault checking
69      // Hold and Suspend conditions are checked before timeout aborts
70      // FaultCode 50 and 60 go to Holding
71      // FaultCode 30 and 40 go to Suspending
72      // Serious faults go to Aborting
73      // ====================================================
```

```
74
75
76      // ========================
77      // Hold type faults
78      // Low fluid should pause the machine
79      // ========================
80
81      IF Low_Fluid_A AND (ContainerType = 1) THEN
82          FaultActive := TRUE;
83          FaultLatched := TRUE;
84          FaultCode := 50;
85          HoldCmd := TRUE;
86      END_IF;
87
88      IF Low_Fluid_B AND (ContainerType = 2) THEN
89          FaultActive := TRUE;
90          FaultLatched := TRUE;
91          FaultCode := 60;
92          HoldCmd := TRUE;
93      END_IF;
94
95
96      // ========================
97      // Suspend type faults
98      // Infeed and outfeed block are treated as temporary external flow problems
99      // ========================
100
101     IF Infeed_Block_Sensor THEN
102         FaultActive := TRUE;
103         FaultLatched := TRUE;
104         FaultCode := 30;
105         SuspendCmd := TRUE;
106     END_IF;
107
108     IF Outfeed_Block_Sensor THEN
109         FaultActive := TRUE;
110         FaultLatched := TRUE;
111         FaultCode := 40;
112         SuspendCmd := TRUE;
113     END_IF;
114
115
116     // ========================
117     // Abort type faults
118     // Serious faults
119     // Timeout faults are disabled while Hold or Suspend is active
120     // ========================
121
122     IF Valve_A AND Valve_B THEN
123         FaultActive := TRUE;
124         FaultLatched := TRUE;
125         FaultCode := 10;
126         AbortCmd := TRUE;
127     END_IF;
128
129     IF Conveyor_Fault THEN
```

```
130         FaultActive := TRUE ;
131         FaultLatched := TRUE ;
132         FaultCode := 20 ;
133         AbortCmd := TRUE ;
134     END_IF ;
135
136     IF T_MoveToFill . Q AND NOT HoldCmd AND NOT SuspendCmd THEN
137         FaultActive := TRUE ;
138         FaultLatched := TRUE ;
139         FaultCode := 70 ;
140         AbortCmd := TRUE ;
141     END_IF ;
142
143     IF T_Fill . Q AND NOT HoldCmd AND NOT SuspendCmd THEN
144         FaultActive := TRUE ;
145         FaultLatched := TRUE ;
146         FaultCode := 80 ;
147         AbortCmd := TRUE ;
148     END_IF ;
149
150     IF T_MoveToExit . Q AND NOT HoldCmd AND NOT SuspendCmd THEN
151         FaultActive := TRUE ;
152         FaultLatched := TRUE ;
153         FaultCode := 90 ;
154         AbortCmd := TRUE ;
155     END_IF ;
156
157     IF T_Pickup . Q AND NOT HoldCmd AND NOT SuspendCmd THEN
158         FaultActive := TRUE ;
159         FaultLatched := TRUE ;
160         FaultCode := 100 ;
161         AbortCmd := TRUE ;
162     END_IF ;
163
164
165     // ====================================================
166     // PackML exit priority logic
167     // Priority order
168     // Abort
169     // Stop
170     // Hold
171     // Suspend
172     // Normal Execute
173     // ====================================================
174
175
176     // ========================
177     // Abort reaction
178     // ========================
179
180     IF Abort_PB OR AbortCmd THEN
181
182         Conveyor_Motor := FALSE ;
183         Valve_A := FALSE ;
184         Valve_B := FALSE ;
185         Pickup_Ready := FALSE ;
```

```
186
187         Alarm_Lamp  :=  TRUE ;
188         Stack_Light_Green  :=  FALSE ;
189         Stack_Light_Amber  :=  FALSE ;
190         Stack_Light_Red  :=  TRUE ;
191
192         ExecStep  :=  0 ;
193
194
195     // ========================
196     // Stop reaction
197     // ========================
198
199     ELSIF  Stop_PB  THEN
200
201         Conveyor_Motor  :=  FALSE ;
202         Valve_A  :=  FALSE ;
203         Valve_B  :=  FALSE ;
204         Pickup_Ready  :=  FALSE ;
205
206         Alarm_Lamp  :=  FALSE ;
207         Stack_Light_Green  :=  FALSE ;
208         Stack_Light_Amber  :=  TRUE ;
209         Stack_Light_Red  :=  FALSE ;
210
211         ExecStep  :=  0 ;
212
213
214     // ========================
215     // Hold reaction
216     // Low fluid conditions go here
217     // SFC should move to Holding
218     // ========================
219
220     ELSIF  HoldCmd  THEN
221
222         Conveyor_Motor  :=  FALSE ;
223         Valve_A  :=  FALSE ;
224         Valve_B  :=  FALSE ;
225         Pickup_Ready  :=  FALSE ;
226
227         Alarm_Lamp  :=  FALSE ;
228         Stack_Light_Green  :=  FALSE ;
229         Stack_Light_Amber  :=  TRUE ;
230         Stack_Light_Red  :=  FALSE ;
231
232
233     // ========================
234     // Suspend reaction
235     // Infeed or outfeed block conditions go here
236     // SFC should move to Suspending
237     // ========================
238
239     ELSIF  SuspendCmd  THEN
240
241         Conveyor_Motor  :=  FALSE ;
```

```
242         Valve_A  :=  FALSE ;
243         Valve_B  :=  FALSE ;
244         Pickup_Ready  :=  FALSE ;
245
246         Alarm_Lamp  :=  FALSE ;
247         Stack_Light_Green  :=  FALSE ;
248         Stack_Light_Amber  :=  TRUE ;
249         Stack_Light_Red  :=  FALSE ;
250
251
252     // ========================
253     // Normal Execute sequence
254     // ========================
255
256     ELSE
257
258         CASE  ExecStep  OF
259
260
261             0 :
262                 // Wait for container at infeed
263                 Conveyor_Motor  :=  FALSE ;
264                 Valve_A  :=  FALSE ;
265                 Valve_B  :=  FALSE ;
266                 Pickup_Ready  :=  FALSE ;
267
268                 ContainerType  :=  0 ;
269
270                 IF  Infeed_Prox  THEN
271                     ExecStep  :=  10 ;
272                 END_IF ;
273
274
275             10 :
276                 // Detect container colour
277                 Conveyor_Motor  :=  FALSE ;
278                 Valve_A  :=  FALSE ;
279                 Valve_B  :=  FALSE ;
280                 Pickup_Ready  :=  FALSE ;
281
282                 IF  Red_Container_Sensor  AND  NOT  Blue_Container_Sensor  THEN
283                     ContainerType  :=  1 ;
284                     ExecStep  :=  20 ;
285
286                 ELSIF  Blue_Container_Sensor  AND  NOT  Red_Container_Sensor  THEN
287                     ContainerType  :=  2 ;
288                     ExecStep  :=  20 ;
289
290                 ELSIF  Red_Container_Sensor  AND  Blue_Container_Sensor  THEN
291                     FaultActive  :=  TRUE ;
292                     FaultLatched  :=  TRUE ;
293                     FaultCode  :=  110 ;
294                     AbortCmd  :=  TRUE ;
295                     ExecStep  :=  0 ;
296
297                 ELSE
```

```
298                     ExecStep := 10 ;
299                 END_IF ;
300
301
302             20 :
303                 // Move container to filling position
304                 Conveyor_Motor := TRUE ;
305                 Valve_A := FALSE ;
306                 Valve_B := FALSE ;
307                 Pickup_Ready := FALSE ;
308
309                 IF Fill_Pos_Sensor THEN
310                     ExecStep := 30 ;
311                 END_IF ;
312
313
314             30 :
315                 // Stop conveyor and choose filling branch
316                 Conveyor_Motor := FALSE ;
317                 Valve_A := FALSE ;
318                 Valve_B := FALSE ;
319                 Pickup_Ready := FALSE ;
320
321                 IF ContainerType = 1 THEN
322                     ExecStep := 40 ;
323
324                 ELSIF ContainerType = 2 THEN
325                     ExecStep := 50 ;
326
327                 ELSE
328                     FaultActive := TRUE ;
329                     FaultLatched := TRUE ;
330                     FaultCode := 120 ;
331                     AbortCmd := TRUE ;
332                     ExecStep := 0 ;
333                 END_IF ;
334
335
336             40 :
337                 // Fill red container with Fluid A
338                 Conveyor_Motor := FALSE ;
339                 Valve_A := TRUE ;
340                 Valve_B := FALSE ;
341                 Pickup_Ready := FALSE ;
342
343                 IF Level_Full_Sensor THEN
344                     ExecStep := 60 ;
345                 END_IF ;
346
347
348             50 :
349                 // Fill blue container with Fluid B
350                 Conveyor_Motor := FALSE ;
351                 Valve_A := FALSE ;
352                 Valve_B := TRUE ;
353                 Pickup_Ready := FALSE ;
```

```
354
355                 IF Level_Full_Sensor THEN
356                     ExecStep := 60 ;
357                 END_IF ;
358
359
360             60 :
361                 // Stop filling
362                 Conveyor_Motor := FALSE ;
363                 Valve_A := FALSE ;
364                 Valve_B := FALSE ;
365                 Pickup_Ready := FALSE ;
366
367                 ExecStep := 70 ;
368
369
370             70 :
371                 // Move filled container to exit
372                 Conveyor_Motor := TRUE ;
373                 Valve_A := FALSE ;
374                 Valve_B := FALSE ;
375                 Pickup_Ready := FALSE ;
376
377                 IF Exit_Sensor THEN
378                     ExecStep := 80 ;
379                 END_IF ;
380
381
382             80 :
383                 // Stop at exit and tell robot to pick up
384                 Conveyor_Motor := FALSE ;
385                 Valve_A := FALSE ;
386                 Valve_B := FALSE ;
387                 Pickup_Ready := TRUE ;
388
389                 ExecStep := 90 ;
390
391
392             90 :
393                 // Wait for robot pickup done signal
394                 Conveyor_Motor := FALSE ;
395                 Valve_A := FALSE ;
396                 Valve_B := FALSE ;
397                 Pickup_Ready := TRUE ;
398
399                 IF Robot_Pick_Done THEN
400
401                     Pickup_Ready := FALSE ;
402                     ContainerType := 0 ;
403
404                     IF BatchDone THEN
405                         ExecStep := 100 ;
406                     ELSE
407                         ExecStep := 0 ;
408                     END_IF ;
409
```

```
410                 END_IF ;
411
412
413             100 :
414                 // Execute completed normally
415                 Conveyor_Motor  :=  FALSE ;
416                 Valve_A  :=  FALSE ;
417                 Valve_B  :=  FALSE ;
418                 Pickup_Ready  :=  FALSE ;
419
420                 SC  :=  TRUE ;
421                 ExecStep  :=  0 ;
422
423
424             ELSE
425                 // Unknown ExecStep value
426                 Conveyor_Motor  :=  FALSE ;
427                 Valve_A  :=  FALSE ;
428                 Valve_B  :=  FALSE ;
429                 Pickup_Ready  :=  FALSE ;
430
431                 FaultActive  :=  TRUE ;
432                 FaultLatched  :=  TRUE ;
433                 FaultCode  :=  999 ;
434                 AbortCmd  :=  TRUE ;
435                 ExecStep  :=  0 ;
436
437         END_CASE ;
438
439     END_IF ;
440
```

1.1.1.9 POU: Held_ST

```
1     FUNCTION_BLOCK  Held_ST
2     VAR_INPUT
3     END_VAR
4     VAR_OUTPUT
5     END_VAR
6     VAR
7     END_VAR
8
```

```
1     // Held
2
3     Conveyor_Motor  :=  FALSE ;
4     Valve_A  :=  FALSE ;
5     Valve_B  :=  FALSE ;
6     Pickup_Ready  :=  FALSE ;
7
8     Stack_Light_Green  :=  FALSE ;
9     Stack_Light_Amber  :=  TRUE ;
10    Stack_Light_Red  :=  FALSE ;
11
12    SC  :=  FALSE ;
```

```
13
```

1.1.1.10 POU: Holding_ST

```
1   FUNCTION_BLOCK Holding_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   // Holding
2
3   Conveyor_Motor := FALSE ;
4   Valve_A := FALSE ;
5   Valve_B := FALSE ;
6   Pickup_Ready := FALSE ;
7
8   Stack_Light_Green := FALSE ;
9   Stack_Light_Amber := TRUE ;
10  Stack_Light_Red := FALSE ;
11
12  SC := TRUE ;
13
```

1.1.1.11 POU: Idle_ST

```
1   FUNCTION_BLOCK Idle_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   Conveyor_Motor := FALSE ;
2   Valve_A := FALSE ;
3   Valve_B := FALSE ;
4   Pickup_Ready := FALSE ;
5
6   Alarm_Lamp := FALSE ;
7   Stack_Light_Green := FALSE ;
8   Stack_Light_Amber := TRUE ;
9   Stack_Light_Red := FALSE ;
10
11  SC := FALSE ;
12
```

1.1.1.12 POU: PLC_PRG

```
PROGRAM PLC_PRG
VAR
END_VAR

```

1.1.1.13 POU: Resetting_ST

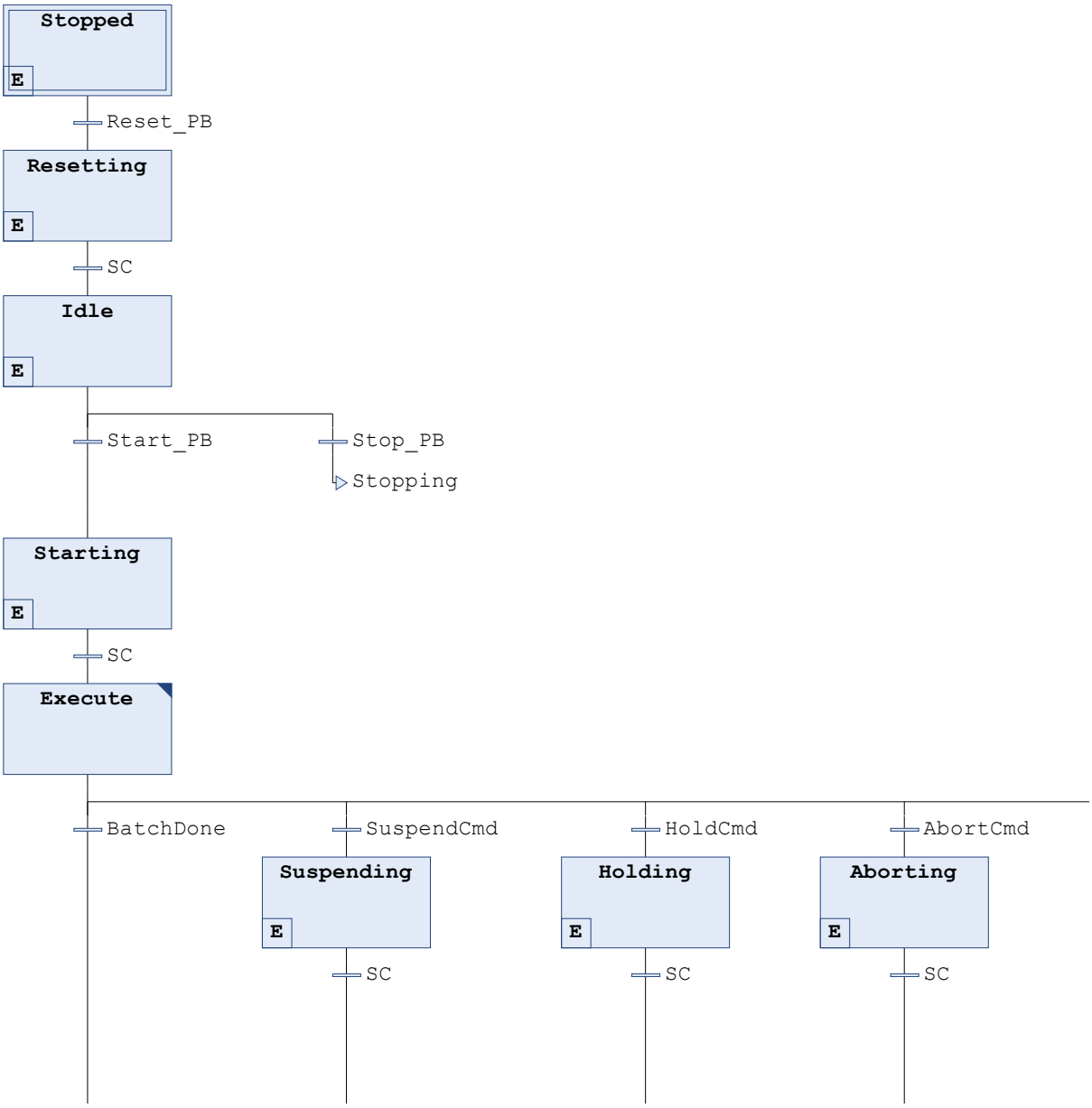
```
FUNCTION_BLOCK Resetting_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

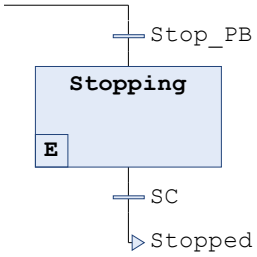
```
BatchDone := FALSE ;
ClearCmd := FALSE ;
HoldCmd := FALSE ;
UnholdCmd := FALSE ;
SuspendCmd := FALSE ;
UnsuspendCmd := FALSE ;

Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := FALSE ;
Stack_Light_Red := FALSE ;

ExecStep := 0 ;

SC := TRUE ;

```

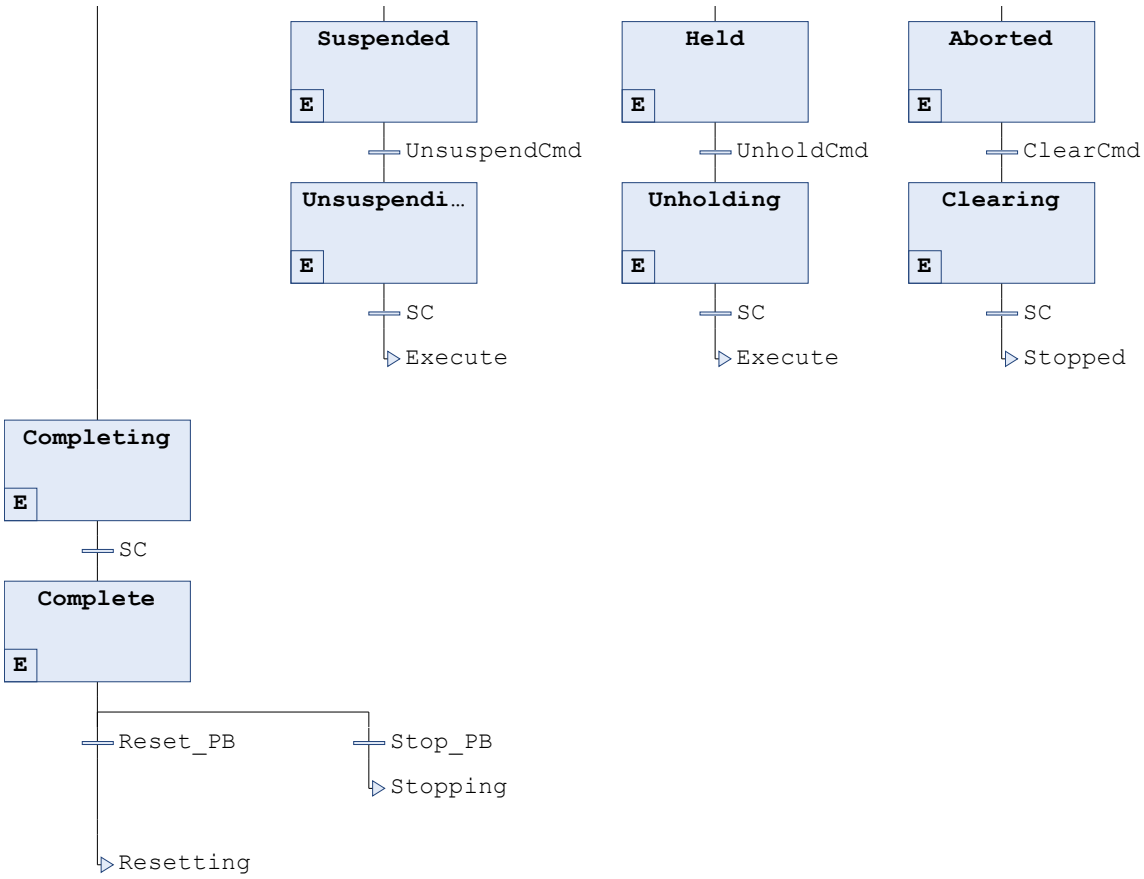
1.1.1.14 POU: sfc

```
PROGRAM sfc
VAR
    fbStopped        : Stopped_ST ;
    fbResetting      : Resetting_ST ;
    fbIdle           : Idle_ST ;
    fbStarting       : Starting_ST ;
    fbExecute        : Execute_ST ;
    fbCompleting     : Completing_ST ;
    fbComplete       : Complete_ST ;
    fbStopping       : Stopping_ST ;
```

```
11          fbAborting       :  Aborting_ST ;
12          fbAborted        :  Aborted_ST ;
13          fbClearing       :  Clearing_ST ;
14          fbHolding        :  Holding_ST ;
15          fbHeld           :  Held_ST ;
16          fbUnholding      :  Unholding_ST ;
17          fbSuspending     :  Suspending_ST ;
18          fbSuspended      :  Suspended_ST ;
19          fbUnsuspending   :  Unsuspending_ST ;
20      END_VAR
21
```




Stop_PB
Stopping
E
SC
Stopped


Suspended
E
UnsuspendCmd
Unsuspendi...
E
SC
Execute
Held
E
UnholdCmd
Unholding
E
SC
Execute
Aborted
E
ClearCmd
Clearing
E
SC
Stopped
Completing
E
SC
Complete
E
Reset_PB
Stop_PB
Stopping
Resetting

1.1.1.14.1 Action: _aAborted_entry

```
fbAborted ( ) ;

```

1.1.1.14.2 Action: _aAborting_entry

```
fbAborting ( ) ;

```

1.1.1.14.3 Action: _aACT_EXECUTE

```
fbExecute ( ) ;

```

1.1.1.14.4 Action: _aClearing_entry

```
fbClearing ( ) ;

```

1.1.1.14.5 Action: _aComplete_entry

```
fbComplete ( ) ;

```

1.1.1.14.6 Action: _aCompleting_entry

```
fbCompleting ( ) ;

```

1.1.1.14.7 Action: _aHeld_entry

```
fbHeld ( ) ;

```

1.1.1.14.8 Action: _aHolding_entry

```
fbHolding ( ) ;

```

1.1.1.14.9 Action: _aIdle_entry

```
fbIdle ( ) ;

```

1.1.1.14.10 Action: _aResetting_entry

```
fbResetting ( ) ;

```

1.1.1.14.11 Action: _aStarting_entry

```
fbStarting ( ) ;

```

1.1.1.14.12 Action: _aStopped_entry

```
fbStopped ( ) ;

```

1.1.1.14.13 Action: _aStopping_entry

```
fbStopping ( ) ;

```

1.1.1.14.14 Action: _aSuspended_entry

```
fbSuspended ( ) ;

```

1.1.1.14.15 Action: _aSuspending_entry

```
fbSuspending ( ) ;

```

1.1.1.14.16 Action: _aUnholding_entry

```
fbUnholding ( ) ;

```

1.1.1.14.17 Action: _aUnsuspending_entry

```
fbUnsuspending ( ) ;

```

1.1.1.15 POU: Starting_ST

```
FUNCTION_BLOCK Starting_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;
```

```
5
6       IF NOT Conveyor_Fault
7          AND NOT Low_Fluid_A
8          AND NOT Low_Fluid_B THEN
9           Alarm_Lamp := FALSE ;
10          Stack_Light_Green := TRUE ;
11          Stack_Light_Amber := FALSE ;
12          Stack_Light_Red := FALSE ;
13          SC := TRUE ;
14      ELSE
15          Alarm_Lamp := TRUE ;
16          Stack_Light_Green := FALSE ;
17          Stack_Light_Amber := FALSE ;
18          Stack_Light_Red := TRUE ;
19          SC := FALSE ;
20      END_IF ;
21
```

1.1.1.16 POU: Stopped_ST

```
1       FUNCTION_BLOCK Stopped_ST
2
```

```
1       Conveyor_Motor := FALSE ;
2       Valve_A := FALSE ;
3       Valve_B := FALSE ;
4       Pickup_Ready := FALSE ;
5
6       Alarm_Lamp := FALSE ;
7       Stack_Light_Green := FALSE ;
8       Stack_Light_Amber := FALSE ;
9       Stack_Light_Red := FALSE ;
10
11      SC := FALSE ;
12
```

1.1.1.17 POU: Stopping

```
1       FUNCTION_BLOCK Stopping
2       VAR_INPUT
3       END_VAR
4       VAR_OUTPUT
5       END_VAR
6       VAR
7       END_VAR
8
```

1.1.1.18 POU: Stopping_ST

```
FUNCTION_BLOCK Stopping_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

SC := TRUE ;

```

1.1.1.19 POU: Suspended_ST

```
FUNCTION_BLOCK Suspended_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
// Suspended
// The machine is paused because of an external condition
// Wait here until UnsuspendCmd is given

Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := TRUE ;
Stack_Light_Red := FALSE ;

// Do not clear SuspendCmd here
// SuspendCmd will be cleared in Unsuspending_ST

SC := FALSE ;

```

1.1.1.20 POU: Suspending_ST

```
FUNCTION_BLOCK Suspending_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
// Suspending
// This state stops the machine because of an external condition
// For example: Infeed_Block_Sensor or Outfeed_Block_Sensor

Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := TRUE ;
Stack_Light_Red := FALSE ;

// Do not clear SuspendCmd here
// Keep SuspendCmd TRUE so that the system stays in the suspend path
// It will be cleared in Unsuspending_ST

SC := TRUE ;

```

1.1.1.21 POU: Unholding_ST

```
FUNCTION_BLOCK Unholding_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
// Unholding
// This state clears the Hold condition and returns to Execute

Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := TRUE ;
Stack_Light_Amber := FALSE ;
```

```
12      Stack_Light_Red  :=  FALSE ;
13
14      // Clear Hold related variables
15      HoldCmd  :=  FALSE ;
16      UnholdCmd  :=  FALSE ;
17
18      // Clear low fluid hold fault
19      FaultActive  :=  FALSE ;
20      FaultLatched  :=  FALSE ;
21      FaultCode  :=  0 ;
22
23      // Keep current Execute step if you want to continue the process
24      // Do not reset ExecStep here unless you want to restart Execute from Step 0
25
26      SC  :=  TRUE ;
27
```

1.1.1.22 POU: Unsuspending_ST

```
1       FUNCTION_BLOCK  Unsuspending_ST
2       VAR_INPUT
3       END_VAR
4       VAR_OUTPUT
5       END_VAR
6       VAR
7       END_VAR
8
```

```
1       // Unsuspending
2       // This state clears the suspend condition and returns to Execute
3
4       Conveyor_Motor  :=  FALSE ;
5       Valve_A  :=  FALSE ;
6       Valve_B  :=  FALSE ;
7       Pickup_Ready  :=  FALSE ;
8
9       Alarm_Lamp  :=  FALSE ;
10      Stack_Light_Green  :=  TRUE ;
11      Stack_Light_Amber  :=  FALSE ;
12      Stack_Light_Red  :=  FALSE ;
13
14      // Clear suspend related commands
15      SuspendCmd  :=  FALSE ;
16      UnsuspendCmd  :=  FALSE ;
17
18      // Clear suspend related fault information
19      FaultActive  :=  FALSE ;
20      FaultLatched  :=  FALSE ;
21      FaultCode  :=  0 ;
22
23      // Do not reset ExecStep here
24      // The system can return to Execute and continue from the current step
25
26      SC  :=  TRUE ;
27
```

1.1.1.23 Task Configuration: Task Configuration

Max. number of tasks: 100
Max. number of cyclic tasks: 100
Max. number of freewheeling tasks: 100
Max. number of event tasks: 100
Max. number of status tasks: 100

System Events:

1.1.1.23.1 Task: MainTask

Priority: 1
Type: Cyclic
Interval: t#20ms Unit: ms
Watchdog: Inactive
POUs: sfc
PLC_PRG

1.1.1.23.1.1 Program Call: sfc

1.1.1.23.1.2 Program Call: PLC_PRG

2 : Project Settings

Static Analysis Light:

Unused variables (#33): 0
Overlapping memory areas (#28): 0
Write access from several tasks (#6): 0
Multiple write access on output (#4): 0
Multiple uses of identifiers (#27): 0
Report temporary FunctionBlock instances (#167): 0
Suspicious operation on string (#175): 0